

Além da Ortogonalidade: Preocupações Constitutivas em Software Quântico

Beyond Orthogonality: Constitutive Concerns in Quantum Software

Jefferson de Oliveira Silva
jefferson.oliveira@prof.inteli.edu.br
Instituto de Tecnologia e Liderança (Inteli)
Pontifícia Universidade Católica de São Paulo (PUC-SP)
São Paulo, Brazil

Murilo Zanini de Carvalho
murilo.zanini@prof.inteli.edu.br
Instituto de Tecnologia e Liderança (Inteli)
São Paulo, Brazil

Bryan Kano Ferreira
bryan.ferreira@prof.inteli.edu.br
Instituto de Tecnologia e Liderança (Inteli)
São Paulo, Brazil

Fabiana Naomi Iegawa
fabiana.iegawa@prof.inteli.edu.br
Instituto de Tecnologia e Liderança (Inteli)
São Paulo, Brazil

Resumo

Programas quânticos podem misturar lógica algorítmica e decisões de execução, como mitigação de erros e adaptação de *hardware*. A programação orientada a aspectos oferece mecanismos para modularizar preocupações transversais, mas essa modularização, por si só, não indica quando sua ausência inviabiliza a execução ou compromete o resultado. Definimos uma preocupação transversal como constitutiva quando, em relação a um circuito, *backend*, condições de ruído, distribuição alvo e limiar de validade, sua remoção impede a execução ou produz resultado abaixo do limiar. Ilustramos a definição em dois casos reproduzíveis. Em uma simulação de Grover com três *qubits* e ruído derivado do *snapshot* FakeFz, a extrapolação para ruído zero elevou a fidelidade da distribuição de 0,953 para 0,977, acima do limiar de 0,97. Na adaptação de *backend*, um circuito de Grover com dois *qubits* e 16 operações lógicas resultou em uma representação com 13 operações nativas após a transpilação para o mesmo alvo. Os casos exemplificam constitutividade do resultado e da executabilidade. Analisamos ainda sua composição com preocupações cuja remoção preserva a executabilidade e o critério de validade, mostrando que a ordem das transformações pode produzir circuitos operacionalmente distintos sob ruído. Por fim, derivamos requisitos para a composição, a implementação e a validação empírica dessas preocupações.

Palavras-chave

Computação Quântica, Engenharia de Software Quântico, Programação Orientada a Aspectos, Preocupações Constitutivas, Mitigação de Erros Quânticos

Abstract

Quantum programs may mix algorithmic logic with execution decisions such as error mitigation and hardware adaptation. Aspect-oriented programming provides mechanisms for modularizing crosscutting concerns, but this modularization alone does not indicate when their absence prevents execution or compromises the result. We define a crosscutting concern as constitutive when, relative to a circuit, backend, noise conditions, target distribution, and validity threshold, removing it prevents execution or produces a result below the threshold. We illustrate the definition through

two reproducible cases. In a three-qubit Grover simulation using noise derived from the FakeFz snapshot, zero-noise extrapolation increased distribution fidelity from 0.953 to 0.977, above the 0.97 threshold. For backend adaptation, a two-qubit Grover circuit containing 16 logical operations resulted in a representation with 13 native operations after transpilation to the same target. The cases illustrate concerns constitutive of result validity and executability. We also analyze their composition with concerns whose removal preserves executability and the validity criterion, showing that transformation order can produce operationally distinct circuits under noise. Finally, we derive requirements for the composition, implementation, and empirical validation of these concerns.

Keywords: Quantum Computing, Quantum Software Engineering, Aspect-Oriented Programming, Constitutive Concerns, Quantum Error Mitigation

1 Introdução

Frameworks como Qiskit [5] e PennyLane [1] reduziram a barreira para escrever programas quânticos ao oferecer interfaces de programação de aplicações (APIs) que abstraem parte dos detalhes físicos do processador e permitem executar algoritmos em *hardware* acessível pela nuvem. Essa abstração simplifica o acesso ao dispositivo, mas não determina como organizar no código as decisões que dependem dele. Mitigação de erros, escolha de *backend* e transpilação ainda precisam ser coordenadas com a lógica algorítmica, o que cria um problema distinto de separação de responsabilidades.

A necessidade dessa separação decorre, em parte, das condições do regime de escala intermediária e ruidosa (*Noisy Intermediate-Scale Quantum*, NISQ), caracterizado por *qubits* físicos sujeitos a ruído e pela ausência de computação tolerante a falhas em larga escala [11]. Escrever um circuito com a lógica correta não é suficiente. O desenvolvedor precisa decidir como mitigar ruído, adaptar o circuito às operações suportadas pelo *hardware* alvo e orquestrar múltiplas execuções para obter estimativas válidas. Essas decisões pertencem à execução, não à lógica do circuito, mas essas APIs permitem expressá-las no mesmo fluxo de código. Essa possibilidade motiva uma investigação sistemática de como separar e classificar tais decisões.

A programação orientada a aspectos (*Aspect-Oriented Programming*, AOP) separa preocupações transversais em módulos declarativos [8]. No domínio quântico, contudo, o modelo de composição não indica quais preocupações são necessárias para a executabilidade ou para satisfazer um critério quantitativo de validade no ambiente alvo. Argumentamos que certas preocupações de execução de *software* quântico exigem essa distinção. Em determinado contexto, sua remoção pode deixar o circuito sem uma representação executável no *hardware* alvo ou fazer a execução ruidosa produzir um resultado abaixo do critério exigido. Propomos o conceito de **preocupação constitutiva** para descrever essa categoria. O termo remete às *equações constitutivas* da física do contínuo, que complementam leis gerais com relações específicas do material.¹ De modo análogo, uma preocupação constitutiva complementa a lógica do circuito com um procedimento necessário no ambiente de execução.

Este artigo apresenta as seguintes contribuições: (1) uma definição formal de preocupação constitutiva baseada na executabilidade e em um critério quantitativo contextual de validade; (2) dois casos ilustrativos: mitigação de erros via extrapolação a ruído zero (*Zero-Noise Extrapolation*, ZNE) e adaptação de *backend*; (3) uma análise da composição entre preocupações ortogonais ao critério de executabilidade e validade e preocupações constitutivas, quanto à ordem de aplicação, à verificação e à exploração de combinações; e (4) uma agenda de formalização e validação da taxonomia proposta.

O restante deste artigo está organizado da seguinte forma. A Seção 2 apresenta os fundamentos de AOP e computação quântica. A Seção 3 formaliza o conceito de preocupação constitutiva. A Seção 4 descreve os dois casos ilustrativos. A Seção 5 analisa a composição entre preocupações ortogonais e constitutivas. A Seção 6 discute trabalhos relacionados. A Seção 7 conclui com uma agenda de pesquisa.

2 Fundamentos

AOP separa preocupações transversais, como controle de acesso, *logging* e monitoramento, em unidades chamadas aspectos. Um aspecto define pontos de junção (*join points*), adiciona comportamento antes ou depois deles (*advice*) e é tecido (*woven*) ao programa em tempo de compilação ou execução [8]. Por exemplo, o decorador hipotético `@ManagersOnly` poderia representar um aspecto de controle de acesso aplicado ao método `deposit()`, restringindo quem pode invocar a operação sem alterar o saldo calculado. Aspectos podem ser relevantes para a correção de um sistema clássico. O modelo de composição, contudo, não os classifica segundo sua necessidade para executar o programa ou satisfazer um limiar quantitativo de validade. A Seção 3 introduz essa distinção para o domínio quântico.

Um *qubit* é um vetor unitário em $\mathbb{C}^2$, $\alpha|0\rangle + \beta|1\rangle$, com $|\alpha|^2 + |\beta|^2 = 1$. Portas quânticas são transformações unitárias, como *H* (Hadamard, usada para criar superposição a partir de $|0\rangle$ ou $|1\rangle$), *X* (análoga ao NOT clássico) e *CZ* (fase condicional entre dois *qubits*, capaz de gerar entrelaçamento dependendo do estado de entrada). Um circuito quântico é uma sequência dessas portas aplicada a um

¹Em mecânica do contínuo, as leis de conservação não determinam, por si sós, o comportamento de um material. Relações constitutivas, como as leis de Hooke, Ohm e Fourier, descrevem esse comportamento ao relacionar, respectivamente, tensão e deformação, corrente e campo elétrico, e fluxo de calor e gradiente de temperatura [13].

registrador. Na medição na base computacional, a regra de Born associa a cada cadeia de bits a probabilidade dada pelo módulo quadrado de sua amplitude [10]. Para um único *qubit* no estado $\alpha|0\rangle + \beta|1\rangle$, as probabilidades são $|\alpha|^2$ e $|\beta|^2$. Como a medição é estocástica, o circuito é executado múltiplas vezes (*shots*) para estimar a distribuição de resultados; dependendo do algoritmo, interessa o valor mais frequente, um valor esperado ou a distribuição inteira.

As distribuições estimadas por essas execuções também são afetadas pelo ruído do dispositivo. No regime NISQ, os *qubits* físicos são sensíveis a perturbações ambientais, que causam decoerência, e os erros das operações dependem do dispositivo e de suas condições de calibração. Circuitos mais profundos tendem a acumular mais erros e degradar os resultados. ZNE [12] é uma técnica de mitigação de erros quânticos: o ruído é amplificado em escalas controladas e os resultados são extrapolados para o limite de ruído zero, sem *qubits* adicionais.

Além de lidar com o ruído, executar um circuito em um dispositivo específico exige adaptar sua representação às operações e à topologia disponíveis. Circuitos são escritos com portas lógicas abstratas, mas cada *chip* quântico suporta apenas um conjunto nativo de operações. O IBM Fez (Heron r2) opera com $\{CZ, RZ, SX, X\}$ [6]; uma porta *H* precisa ser decomposta nessa família. Além disso, a topologia do *chip* determina quais *qubits* físicos podem interagir diretamente. Quando a lógica exige interação entre *qubits* não adjacentes, operações de roteamento, como SWAP, podem ser inseridas pelo transpilador, aumentando a profundidade e o erro acumulado. O mesmo circuito lógico produz circuitos nativos distintos dependendo do *backend* alvo. Embora compilações clássicas também possam apresentar diferenças dependentes da plataforma, por exemplo em aritmética de ponto flutuante, na transpilação quântica a seleção de portas, o mapeamento e o ruído do dispositivo influenciam a distribuição observada.

3 Preocupações Constitutivas

A Listagem 1 apresenta, em notação Python inspirada na API do Qiskit, uma forma declarativa de separar a lógica do algoritmo de Grover [4] das decisões de execução. O corpo da função expressa apenas a lógica do circuito; as decisões de mitigação e adaptação de *hardware* são representadas pelos decoradores.

Os decoradores usados neste artigo, incluindo `@ManagersOnly`, `@ErrorMitigation`, `@BackendAdapt`, `@Logging`, `@CostMonitor` e `@Explore`, constituem uma notação conceitual, e não APIs existentes do Qiskit ou de outro *framework*. Empregamos a sintaxe de decoradores Python por sua familiaridade e por explicitar a separação declarativa proposta. O artefato reproduz as transformações e os experimentos subjacentes; sua integração em um *framework* de decoradores poderá oferecer diretamente essa interface sobre SDKs quânticos.

A notação torna a separação visível, mas não determina se cada preocupação pode ser removida sem comprometer a executabilidade ou a validade do resultado. Para responder a essa questão, precisamos primeiro definir o que constitui um resultado válido em determinado contexto de execução.

DEFINIÇÃO 1 (VALIDADE SEMÂNTICA). *Seja Q um circuito quântico e $\mathcal{E}(Q; B, \eta_B)$ um procedimento de execução ou estimativa que*

```

1 @ErrorMitigation(strategy="ZNE", scale_factors
  = [1, 2, 3])
2 @BackendAdapt(target="ibm_fez",
  optimization_level=3)
3 def grover_search(target: int):
4     qc = QuantumCircuit(2)
5     qc.h(0); qc.h(1)
6     for q in range(2):
7         if not (target & (1 << q)): qc.x(q)
8     qc.cz(0, 1)
9     for q in range(2):
10        if not (target & (1 << q)): qc.x(q)
11    qc.h(0); qc.h(1); qc.x(0); qc.x(1)
12    qc.cz(0, 1)
13    qc.x(0); qc.x(1); qc.h(0); qc.h(1)
14    return qc

```

Listagem 1: Notação proposta para separar a lógica do circuito de preocupações de execução.

retorna uma distribuição de probabilidade no backend B sob condições de ruído descritas por η_B . Dadas uma distribuição alvo $\mathcal{D}^*$, uma função de fidelidade F e uma tolerância $0 < \tau \leq 1$ fixada pelo caso de uso, o procedimento é semanticamente válido em relação a $(Q, B, \eta_B, \mathcal{D}^*, \tau)$ se $F(\mathcal{E}(Q; B, \eta_B), \mathcal{D}^*) \geq \tau$.

Adotamos a fidelidade clássica entre distribuições, $F(P, R) = (\sum_i \sqrt{P_i R_i})^2$, cujo valor máximo é 1. Outras métricas também são compatíveis com a Definição 1. Para algoritmos em que o critério de sucesso é um valor esperado, F pode ser substituída por uma função de adequação na qual valores maiores indicam melhores resultados. Se for empregada uma distância Δ , a condição deve ser escrita como $\Delta(\mathcal{E}(Q; B, \eta_B), \mathcal{D}^*) \leq \tau_\Delta$. O descritor η_B pode representar vários parâmetros e variar ao longo do tempo; ele não presuppõe um único parâmetro escalar de ruído.

Neste trabalho, os termos *transversal* e *constitutiva* respondem a perguntas distintas. O primeiro qualifica a distribuição da preocupação na decomposição do programa; o segundo, a consequência de removê-la em um contexto de execução. Nesse segundo eixo, uma preocupação transversal pode ser ortogonal ao critério de executabilidade e validade ou constitutiva em relação a esse critério.

Uma preocupação transversal é ortogonal ao critério no contexto $(Q, B, \eta_B, \mathcal{D}^*, \tau)$ quando sua remoção preserva a executabilidade e a validade, ainda que possa alterar estrutura, custo ou comportamento observável. A classificação, portanto, não decorre da sintaxe do decorador nem da ausência de efeitos sobre o programa, mas da consequência de sua remoção sobre o critério definido.

DEFINIÇÃO 2 (PREOCUPAÇÃO CONSTITUTIVA). Uma preocupação transversal C é constitutiva em relação a $(Q, B, \eta_B, \mathcal{D}^*, \tau)$ se o procedimento com a preocupação, $\mathcal{E}_C$, é executável e satisfaz $F(\mathcal{E}_C(Q; B, \eta_B), \mathcal{D}^*) \geq \tau$. Sua remoção resulta em um procedimento $\mathcal{E}_{-C}$ que (a) não é executável em B ; ou (b) é executável, mas satisfaz $F(\mathcal{E}_{-C}(Q; B, \eta_B), \mathcal{D}^*) < \tau$.

A mitigação de erros via ZNE ilustra a condição (b). A listagem emprega 2 qubits para manter concisa a apresentação da notação. A avaliação de ZNE emprega uma busca de Grover sobre 3 qubits (2 iterações, oráculo e difusor com porta Toffoli, ou CCX). Após

a transpilação, o circuito contém 191 operações na base nativa, o que permite comparar os procedimentos sob os efeitos do modelo de ruído em relação ao limiar adotado². Para este exemplo, adotamos $\tau = 0,97$. Sob o mesmo η_B , $\mathcal{E}_{-C}$ retorna $F \approx 0,953$, enquanto $\mathcal{E}_C$ retorna $F \approx 0,977$ após ZNE; o simulador sem ruído fornece a distribuição de referência. A distribuição mitigada é construída extrapolando separadamente cada probabilidade, truncando valores negativos e normalizando o vetor. Assim, $0,953 < \tau \leq 0,977$. A extrapolação direta da fidelidade produziria $F \approx 0,987$, mas é apenas diagnóstica: exige acesso à distribuição ideal e não reconstrói uma distribuição de saída.

A adaptação de *backend* ilustra a condição (a). O circuito da Listagem 1 utiliza a porta H , que não pertence ao conjunto nativo do IBM Fez. Sem a transformação representada por `@BackendAdapt`, seja ela invocada explicitamente ou realizada de modo implícito pela plataforma, o circuito não pode ser executado no *backend* alvo; o problema é realizabilidade física, não otimização.

Os exemplos anteriores também ajudam a delimitar o alcance do conceito. Preocupação constitutiva não é sinônimo de requisito de execução. Um requisito é uma pré-condição e não diz, por si só, como modularizar o procedimento que o satisfaz. Preocupações de categorias semânticas distintas podem ser expressas pelo mesmo mecanismo sintático. As Listagens 1 e 2 representam preocupações constitutivas e ortogonais como decoradores, mas a sintaxe não codifica a consequência de removê-las. A classificação torna explícita a relação de cada preocupação com a executabilidade ou com um critério operacional de validade.

A Tabela 1 sintetiza esse critério e mostra que alterações estruturais ou múltiplas execuções, isoladamente, não determinam a classificação.

4 Instâncias de Preocupações Constitutivas

A seção anterior empregou os dois casos para ilustrar as condições da Definição 2. Nesta seção, detalhamos a intenção, a interface declarativa e a sequência de transformações de cada preocupação. Sua classificação continua baseada na consequência da remoção, e não na sintaxe do decorador.

4.1 Instância 1: Mitigação de Erros

A notação `@ErrorMitigation` representa um procedimento de estimativa que separa a mitigação de erros da lógica do circuito. No contexto avaliado, esse procedimento permite satisfazer o critério de validade definido para a execução ruidosa.

Na abstração proposta, `@ErrorMitigation` recebe *strategy*, que seleciona o método de mitigação, como ZNE, cancelamento probabilístico de erros (*Probabilistic Error Cancellation*, PEC) ou regressão baseada em dados de circuitos (*Clifford Data Regression*, CDR). Quando a estratégia selecionada é ZNE, *scale_factors* especifica as escalas nominais empregadas na extrapolação. Para a estratégia ZNE executada no artefato:

²Simulação realizada no qiskit-aer com o modelo de ruído do FakeFez (qiskit-ibm-runtime), sem execução em um processador quântico físico. Foram usados 32.768 shots por escala, semente de transpilação 6 e sementes de simulação 54.321, 54.322 e 54.323. O folding global do Mitiq [9], com fatores nominais [1, 2, 3], resultou em circuitos com 191, 815 e 881 operações na base nativa. Os fatores indicam o nível de folding, não múltiplos exatos da contagem de operações ou do ruído físico. As probabilidades foram extrapoladas individualmente com numpy.polyfit.

Tabela 1: Classificação relativa de preocupações transversais ortogonais ao critério e constitutivas. Alterações estruturais e número de execuções não determinam a categoria.

Dimensão	Ortogonal ao Critério	Preocupação Constitutiva
Consequência da remoção	Preserva execução e limiar	Viola execução ou limiar
Dependência de contexto	Relativa a $Q, B, \eta_B, \mathcal{D}^*, \tau$	Relativa aos mesmos elementos
Estrutura do circuito	Pode ser alterada	Pode ser alterada
Número de execuções	Uma ou mais	Uma ou mais
Papel da classificação	Opcional para o critério	Necessária para o critério

- (1) O circuito base é transpilado para o conjunto de portas nativas do *backend*.
- (2) O circuito de escala $\lambda = 1$ permanece inalterado. Para $\lambda > 1$, o *folding* global é aplicado ao circuito transpilado, inserindo pares inversos $U^\dagger U$ no circuito inteiro e, quando necessário, em uma fração dele, preservando a unidade ideal enquanto amplia nominalmente o ruído [3].
- (3) As operações inversas introduzidas pelo *folding* são traduzidas novamente para a base nativa, sem otimizações que poderiam cancelar os pares inseridos.
- (4) Um circuito é executado independentemente para cada fator de escala, com o número configurado de *shots*.
- (5) Para cada resultado, suas probabilidades nas diferentes escalas são ajustadas por um modelo linear em λ , avaliado em $\lambda = 0$. Os valores negativos são truncados, e o vetor resultante é normalizado para formar a distribuição mitigada.

O argumento segue da Definição 2, condição (b). No exemplo, $\mathcal{E}_C$ não satisfaz o limiar, enquanto $\mathcal{E}_C$ constrói, a partir das distribuições nas três escalas, uma estimativa que o satisfaz sob o mesmo η_B . Os circuitos escalonados constituem execuções adicionais introduzidas exclusivamente pelo procedimento de mitigação. Sem as distribuições obtidas nessas escalas, não é possível construir a estimativa que, nesta instância, satisfaz o limiar. A preocupação é, portanto, constitutiva do resultado.

4.2 Instância 2: Adaptação de Backend

A notação @BackendAdapt representa a transformação da representação do circuito para o *backend* selecionado, sem alterar a computação lógica pretendida.

Na abstração proposta, @BackendAdapt recebe *target*, o identificador do *backend*, e *optimization_level*, que define o nível de otimização empregado na transpilação (0 a 3). A transformação compreende:

- (1) **Decomposição de portas:** portas lógicas não suportadas pelo *backend* são decompostas em operações nativas. Para o IBM Fez, com conjunto $\{CZ, RZ, SX, X\}$, CZ já é nativa; H é equivalente, salvo fase global, a uma composição de RZ e SX . Para *target*=0, as 16 operações lógicas da Listagem 1 resultam em 13 operações na base nativa após otimização³.

- (2) **Mapeamento de qubits:** qubits lógicos são mapeados a qubits físicos selecionados segundo a conectividade e as propriedades de calibração registradas no alvo.
- (3) **Roteamento:** quando a lógica exige interação entre qubits físicos não adjacentes, operações SWAP podem ser inseridas. Uma SWAP equivale logicamente a três portas CX (controlled- X), que ainda precisam ser decompostas caso não pertençam à base nativa.
- (4) **Otimização:** sequências redundantes são canceladas; portas RZ adjacentes são fundidas; portas $RZ(0)$ são eliminadas.

O argumento de constitutividade segue da Definição 2, condição (a): sem a transformação representada por @BackendAdapt, o circuito contém instruções que não pertencem ao conjunto aceito pelo dispositivo alvo. O problema não é apenas a qualidade da otimização, mas a ausência de uma representação executável naquele *backend*. Embora essa transformação corresponda a uma etapa de compilação, tratá-la como preocupação declarativa permite reconfigurar o alvo para o mesmo corpo algorítmico e reutilizar a decisão em diferentes algoritmos; outro *backend* pode produzir uma representação nativa diferente.

4.3 Taxonomia Preliminar

As duas instâncias ilustram modos distintos de constitutividade. A mitigação de erros é constitutiva do *resultado* no contexto avaliado: sem ela, $\mathcal{E}_C$ não satisfaz τ . A adaptação de *backend* é constitutiva da *executabilidade*: sem ela, a computação não pode ocorrer no alvo. Essa distinção sugere pelo menos duas categorias. O orçamento de *shots* e outras configurações de execução são candidatos a novas instâncias, cuja classificação dependerá de sua relação com a executabilidade ou com o critério de validade.

5 Composição de Preocupações Ortogonais ao Critério e Constitutivas

As seções anteriores trataram preocupações ortogonais ao critério e constitutivas isoladamente. Na prática, o fluxo de execução de um circuito quântico pode combinar ambas:

Essa composição suscita três questões para o modelo proposto.

5.1 A Ordem das Transformações é Semanticamente Relevante

No AOP clássico, a ordem pode ser relevante quando aspectos interagem, e mecanismos de precedência permitem ordená-los. Esses mecanismos, contudo, não expressam por si sós que determinada

³Com Qiskit 2.4, FakeFez, *optimization_level*=3 e semente de transpilação 6, o circuito transpilado contém seis operações *sx*, cinco *rz* e duas *cz*. A contagem considera instruções do circuito transpilado; uma operação *rz* pode corresponder a uma rotação virtual.

```

1 # preocupacoes ortogonais
2 @Logging(level="INFO")
3 @CostMonitor(budget=100)
4 # constitutivo do resultado
5 @ErrorMitigation(strategy="ZNE",
6                  scale_factors=[1, 2, 3])
7 # constitutivo da executabilidade
8 @BackendAdapt(target="ibm_fez",
9               optimization_level=3)
10 def grover_search(target: int):
11     ...

```

Listagem 2: Composição de preocupações ortogonais e constitutivas via decoradores.

ordem é necessária para satisfazer um limiar quantitativo definido pelo domínio. No domínio quântico, a ordem das transformações constitutivas pode alterar o circuito nativo produzido e, sob ruído, a distribuição de saída.

Considerando a ordem das transformações, aplicar a mitigação de erros antes da adaptação de *backend* produz circuitos escalonados em portas lógicas, posteriormente transpilados de forma individual; cada circuito pode resultar em um mapeamento distinto. Aplicar a adaptação de *backend* antes da mitigação, como no experimento deste trabalho, transpila o circuito base uma vez e aplica o *folding* às operações resultantes. Embora preservem a computação lógica ideal, as duas ordens produzem circuitos estruturalmente diferentes e não são operacionalmente equivalentes sob ruído.

Em Python, a expressão com decoradores empilhados é construída de baixo para cima. Na Listagem 2, `@BackendAdapt` envolve primeiro a função e `@ErrorMitigation` envolve o resultado. A ordem efetiva das transformações durante uma chamada, porém, depende do protocolo implementado pelos decoradores. A notação precisa, portanto, definir essa semântica explicitamente.

Essa diferença estabelece como requisito uma semântica de composição capaz de representar e verificar restrições de precedência entre preocupações. A política de composição entre preocupações constitutivas, assim como sua interação com preocupações ortogonais, integra a agenda da Seção 7.

5.2 Constitutivos como Restrição de Tipo

Mecanismos de AOP permitem adicionar e remover aspectos independentemente, mas não registram quais transformações são necessárias para um ambiente alvo. A classificação proposta torna essa necessidade explícita: um circuito sem `@BackendAdapt` não pode ser executado, e um circuito sem mitigação pode produzir uma distribuição que não satisfaz o limiar de validade.

Isso sugere uma analogia com sistemas de tipos: assim como uma configuração incompatível com as regras de tipo deve ser rejeitada, uma configuração sem as preocupações constitutivas exigidas pelo alvo e pelo critério de validade não deveria ser aceita como válida. A ausência de `@BackendAdapt` pode impedir a execução, enquanto a ausência de mitigação pode produzir um resultado abaixo do limiar estabelecido. Um *framework* precisaria distinguir preocupações ortogonais de constitutivas. A executabilidade pode admitir verificações estáticas, mas a condição quantitativa depende de $\mathcal{D}^*$, η_B e evidência experimental; sua verificação pode exigir contratos,

```

1 @Explore(
2     backends=["ibm_fez", "ionq_aria"],
3     strategies=["ZNE", "PEC", "CDR"]
4 )
5 def grover_search(target: int):
6     ...

```

Listagem 3: Exploração automática de combinações de backend e estratégia de mitigação.

limites analíticos, calibração ou análise dinâmica, e não apenas tipos estáticos.

5.3 Exploração Automática de Combinações

A separação entre lógica algorítmica e preocupações constitutivas permite representar a exploração de configurações como uma operação explícita do modelo: dado um algoritmo fixo, um *framework* poderia explorar combinações de *backends* e estratégias de mitigação:

Sem um mecanismo declarativo, a parametrização e a orquestração das seis combinações do exemplo permaneceriam externas ao corpo algorítmico. Com a separação proposta, o *framework* centralizaria essa variabilidade e poderia selecionar a combinação que maximiza $F(\mathcal{E}(Q; B, \eta_B), \mathcal{D}^*)$. A disponibilidade de $\mathcal{D}^*$ é plausível em *benchmarks*, mas não em todo problema de produção; a definição de critérios substitutos integra a agenda de pesquisa.

6 Trabalhos Relacionados

Na literatura de padrões para *software* quântico, Weigold et al. descrevem soluções recorrentes para codificação, preparação e processamento de dados, como *Angle Encoding*, *Schmidt Decomposition* e *Quantum Phase Estimation* [15]. Em um nível arquitetural, Weder et al. modelam aplicações híbridas por meio da orquestração do fluxo de controle e dados e da orquestração do provisionamento da topologia de execução [14]. Esses trabalhos estruturam algoritmos e aplicações quânticas, enquanto nossa classificação examina preocupações transversais pela consequência de sua remoção.

AspectJulia [7] implementa AOP como um pacote para Julia por meio de metaprogramação e manipulação da árvore sintática abstrata. O trabalho apresenta preocupações como *logging*, *caching* e paralelização em aplicações científicas e de computação de alto desempenho (*High-Performance Computing*, HPC). Essas preocupações não são classificadas em função de um *backend*, uma distribuição alvo e um limiar quantitativo. O presente trabalho acrescenta esse critério contextual.

Na área de mitigação de erros, Mitiq [9] oferece um conjunto extensivo de métodos, incluindo ZNE, PEC e CDR, compatível com diferentes *backends* e *frameworks*. Qermit [2] adota um projeto modular baseado em grafos para compor protocolos e sub-rotinas de mitigação. Esses trabalhos demonstram como procedimentos de mitigação podem ser modularizados e compostos. Nossa análise acrescenta outra questão: em que contexto a remoção de um desses procedimentos viola a executabilidade ou o limiar de validade e, portanto, torna a preocupação constitutiva?

A verificação formal oferece outro ponto de contato. Ying [16] desenvolve uma lógica de Floyd–Hoare para correção parcial e total de programas quânticos, baseada em predicados quânticos e em pré-condições mais fracas. A formulação citada não incorpora o modelo contextual de ruído e as restrições de *hardware* representados por B e η_B na Definição 2.

Esses trabalhos atuam em níveis complementares: os catálogos de padrões estruturam algoritmos e aplicações; AspectJulia modulariza preocupações transversais; Mitiq e Qermit modularizam procedimentos de mitigação; e a lógica de Floyd–Hoare formaliza propriedades de correção de programas quânticos. Este trabalho relaciona essas dimensões ao classificar preocupações transversais segundo o efeito de sua remoção sobre a executabilidade e a validade contextual.

7 Agenda de Pesquisa e Conclusão

O conceito de preocupação constitutiva e a análise de composição da Seção 5 motivam quatro linhas de investigação.

A primeira é a **formalização da semântica de composição**. A Seção 5 mostra que a ordem das transformações pode afetar a validade do resultado e que a presença e a precedência das preocupações constitutivas podem exigir verificações estáticas, dinâmicas ou híbridas. Uma formalização poderia estender a lógica de Hoare quântica examinada [16] para incorporar ruído e restrições de *hardware*, além de definir uma álgebra de composição entre preocupações.

A segunda é de **implementação**. Um *framework* de decoradores constitutivos para kits de desenvolvimento de *software* (*Software Development Kits*, SDKs), como Qiskit e PennyLane, deveria (a) verificar condições de executabilidade e combinar contratos, calibração ou evidência dinâmica para avaliar critérios de validade, (b) impor restrições de precedência entre transformações e (c) explorar combinações de *backends* e estratégias de mitigação. Preocupações que geram múltiplas execuções, como ZNE, exigem uma semântica de tecimento capaz de representar cada execução e a agregação de seus resultados.

A terceira é **empírica**. Com que frequência preocupações de execução aparecem misturadas à lógica algorítmica em *software* quântico? A mineração de repositórios públicos poderia quantificar essa ocorrência e caracterizar suas formas. Entrevistas ou questionários poderiam investigar se os desenvolvedores percebem essa mistura como um problema de projeto e como procuram resolvê-la.

A quarta é de **taxonomia**. As instâncias apresentadas sustentam duas categorias preliminares: constitutividade do resultado e constitutividade da executabilidade. Orçamento de *shots*, configurações de execução e variação temporal do *hardware* são candidatas a novas instâncias. Sua análise permitirá avaliar se as duas categorias são suficientes ou se a taxonomia precisa ser ampliada. Essa linha inclui investigar como determinar τ a partir de requisitos da aplicação, referências experimentais e incerteza estatística. Como a classificação depende desse limiar, diferentes critérios para estabelecê-lo podem alterar se uma preocupação é ortogonal ao critério ou constitutiva em determinado contexto. A classificação deve permanecer baseada na Definição 2, e não apenas em semelhanças entre exemplos.

Escopo da avaliação. A avaliação abrange duas instâncias da taxonomia em um ambiente reproduzível: ZNE sob um modelo de ruído derivado do FakeFz e adaptação para uma arquitetura supercondutora. As estimativas são condicionadas ao modelo, às sementes e ao número de *shots*, e $\tau = 0,97$ expressa o critério operacional do caso estudado. A distribuição $\mathcal{D}^*$ está disponível neste *benchmark*; em tarefas nas quais ela é desconhecida, a avaliação de validade pode recorrer a critérios verificáveis, como propriedades parciais, limites analíticos ou evidência experimental. Esse desenho estabelece uma base reproduzível para ampliar a validação em *hardware* ao vivo, em outras plataformas e com esses critérios.

AOP oferece mecanismos para modularizar preocupações transversais. A classificação proposta acrescenta a distinção entre preocupações cuja remoção preserva a executabilidade e o critério de validade e aquelas cuja remoção os viola. Essa distinção é particularmente relevante na computação quântica, em que os resultados são probabilísticos e dependem das condições do *hardware*. Neste trabalho, definimos preocupação constitutiva, ilustramos a definição com mitigação de erros e adaptação de *backend* e analisamos sua composição com preocupações ortogonais. Esses resultados estabelecem a base conceitual para as linhas de investigação apresentadas nesta seção.

Disponibilidade de Artefatos

A implementação modular dos exemplos de ZNE e adaptação de *backend*, acompanhada de dependências fixadas, testes, resultados de referência e um *notebook* executável, está disponível em <https://github.com/j3ffsilva/quantum-constitutive-concerns>. O artefato distingue o código executável da notação conceitual dos decoradores apresentada nas Listagens 1 a 3.

Referências

- [1] Ville Bergholm, Josh Izaac, Maria Schuld, Christian Gogolin, M. Sohaib Alam, Shah Nawaz Ahmed, Juan Miguel Arrazola, Carsten Blank, Alain Delgado, Soran Jahangiri, Keri McKiernan, Johannes Jakob Meyer, Zeyue Niu, Antal Száva, and Nathan Killoran. 2018. PennyLane: Automatic Differentiation of Hybrid Quantum-Classical Computations. *arXiv:1811.04968*.
- [2] Cristina Cirstoiu, Silas Dilkes, Daniel Mills, Seyon Sivarajah, and Ross Duncan. 2023. Volumetric Benchmarking of Error Mitigation with Qermit. *Quantum* 7 (2023), 1059. doi:10.22331/q-2023-07-13-1059
- [3] Tudor Giurgica-Tiron, Yousef Hindy, Ryan LaRose, Andrea Mari, and William J. Zeng. 2020. Digital Zero Noise Extrapolation for Quantum Error Mitigation. In *2020 IEEE International Conference on Quantum Computing and Engineering (QCE)*. IEEE, 306–316. doi:10.1109/QCE49297.2020.00045
- [4] Lov K. Grover. 1996. A Fast Quantum Mechanical Algorithm for Database Search. In *Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC)*. ACM, 212–219. doi:10.1145/237814.237866
- [5] IBM Quantum. 2023. Qiskit: An Open-source Framework for Quantum Computing. <https://qiskit.org>
- [6] IBM Quantum. 2024. IBM Quantum Platform. <https://quantum.ibm.com>
- [7] Osamu Ishimura and Yoshihide Yoshimoto. 2024. Aspect-Oriented Programming with Julia. *arXiv:2412.18885*; also publ. IEEE Xplore (doc. 11078356).
- [8] Gregor Kiczales, John Lamping, Anurag Mendhekar, Chris Maeda, Cristina Lopes, Jean-Marc Loingtier, and John Irwin. 1997. Aspect-Oriented Programming. In *Proceedings of the European Conference on Object-Oriented Programming (ECOOP)*. Springer, 220–242. doi:10.1007/BFb0053381
- [9] Ryan LaRose, Andrea Mari, Sarah Kaiser, Peter D. Karalekas, Andre A. Alves, Piotr Czarnik, Mohamed El Mandouh, Max H. Gordon, Yousef Hindy, Alan Robertson, Purva Thakre, Misty Wahl, Danny Samuel, Rahul Mistri, Maxime Tremblay, Nick Gardner, Nathaniel T. Stemen, Nathan Shammah, and William J. Zeng. 2022. Mitiq: A Software Package for Error Mitigation on Noisy Quantum Computers. *Quantum* 6 (2022), 774. doi:10.22331/q-2022-08-11-774
- [10] Michael A. Nielsen and Isaac L. Chuang. 2010. *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press, Cambridge. doi:10.1017/CBO9780511976667

- [11] John Preskill. 2018. Quantum Computing in the NISQ Era and Beyond. *Quantum* 2 (2018), 79. doi:10.22331/q-2018-08-06-79
- [12] Kristan Temme, Sergey Bravyi, and Jay M. Gambetta. 2017. Error Mitigation for Short-Depth Quantum Circuits. *Physical Review Letters* 119, 18 (2017). doi:10.1103/PhysRevLett.119.180509
- [13] Clifford Truesdell and Walter Noll. 2004. *The Non-Linear Field Theories of Mechanics* (3 ed.). Springer. doi:10.1007/978-3-662-10388-3
- [14] Benjamin Weder, Johanna Barzen, Frank Leymann, and Michael Zimmermann. 2021. Hybrid Quantum Applications Need Two Orchestrations in Superposition: A Software Architecture Perspective. In *Proceedings of the 18th IEEE International Conference on Web Services (ICWS)*. IEEE. doi:10.1109/ICWS53863.2021.00015
- [15] Manuela Weigold, Johanna Barzen, Frank Leymann, and Marie Salm. 2021. Encoding Patterns for Quantum Algorithms. *IET Quantum Communication* 2, 4 (2021), 141–152. doi:10.1049/qtc2.12032
- [16] Mingsheng Ying. 2012. Floyd–Hoare Logic for Quantum Programs. *ACM Transactions on Programming Languages and Systems* 33, 6 (2012), 19:1–19:49. doi:10.1145/2049706.2049708